User Disconnection Handling (Heartbeat Mechanism)

In real-world systems, internet connectivity is not always stable. Mobile users frequently move between Wi-Fi, cellular networks, tunnels, elevators, or weak coverage zones.

For applications like:

```
None
Chat Apps

WhatsApp / Telegram

Slack / Discord

Gaming Systems

Live Collaboration Apps
```

the system must correctly detect whether a user is **online or offline**.

1. Problem Statement

A persistent connection (WebSocket / TCP) may break unexpectedly.

Naive approach:

```
None
Connection lost → mark user offline

Reconnect → mark user online
```

This creates problems.

2. Why Naive Approach Fails

Users may disconnect briefly due to:

```
None
Network fluctuation

Tunnel travel

Switching Wi-Fi to mobile data

Temporary packet loss

Phone background mode

Battery optimization
```

Result:

```
None
Online → Offline → Online → Offline
```

Presence indicator keeps flickering.

This creates poor user experience.

3. Better Solution: Heartbeat Mechanism

Instead of immediately marking offline, use periodic heartbeat signals.

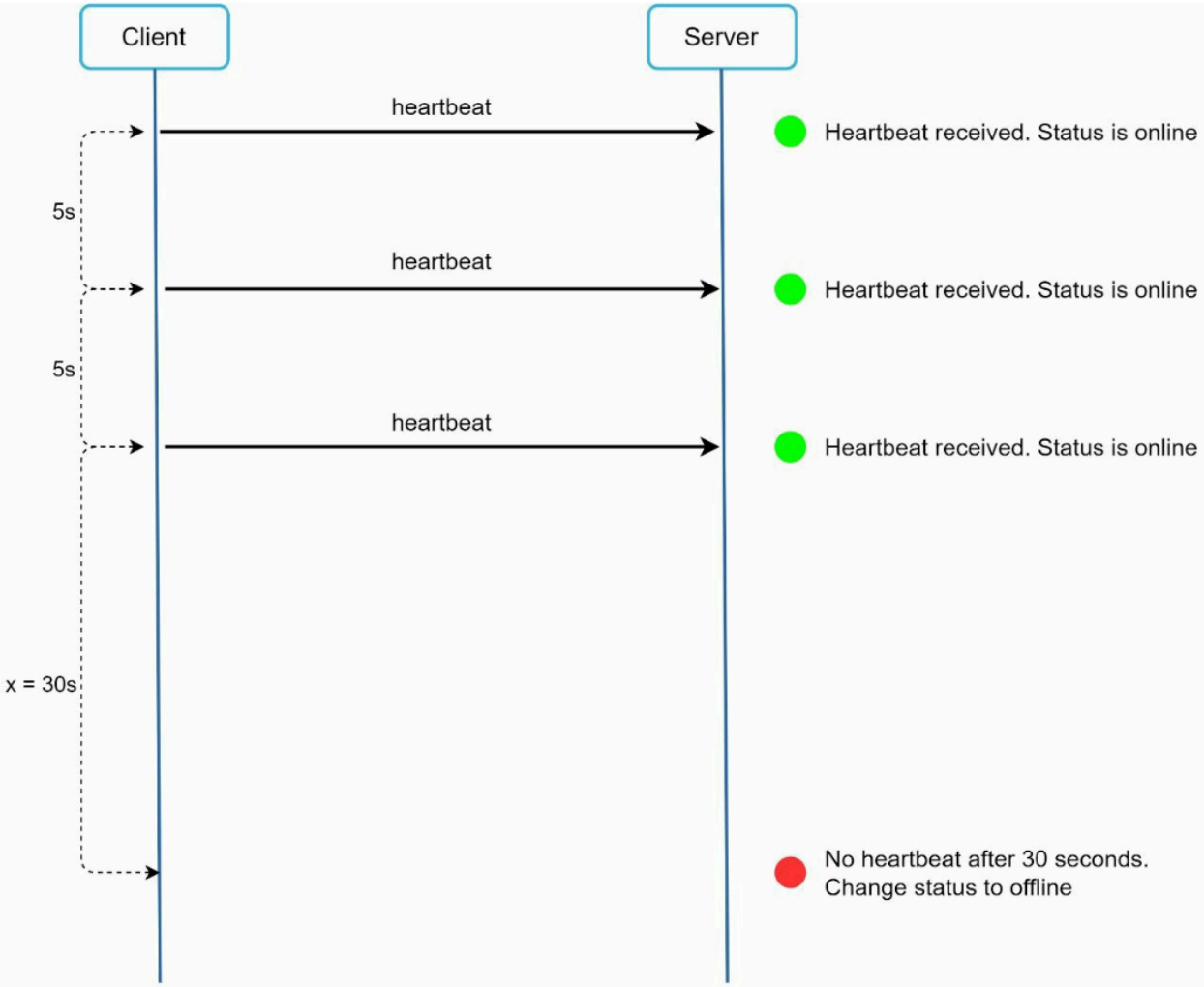
```
None
Client sends heartbeat every few seconds

Server expects heartbeat within timeout window
```

If heartbeat continues:

```
None
User = Online
```

If heartbeat missing for threshold time:

```
None
User = Offline
```



4. How Heartbeat Works

Example Values

None

```
Heartbeat interval = 5 seconds

Offline timeout = 30 seconds
```

Flow

None

```
t=0   heartbeat received

t=5   heartbeat received

t=10  heartbeat received

t=15  no heartbeat

t=20  no heartbeat

t=25  no heartbeat

t=30  timeout exceeded


User marked OFFLINE
```

5. Architecture

```
None
Mobile App / Browser
        ↓
   WebSocket Connection
        ↓
 Presence Server
        ↓
 Redis / Memory Store
        ↓
 Online Status DB
```

6. Presence Server Responsibilities

Presence service manages:

```
None
Track active sockets

Receive heartbeat ping/pong

Maintain last_seen timestamp

Mark online/offline

Broadcast presence changes
```

7. Data Model

```
None
user_id: 501


status: online
last_seen: 10:45:22
device_count: 2
```

8. Algorithm

On Heartbeat Receive

```
None
last_seen = current_time
status = online
```

Background Checker Runs Periodically

```
None
if current_time - last_seen > timeout:
    status = offline
```

9. Why This Is Better

✔ Avoids Flickering Status

Short disconnects do not instantly mark offline.

✔ Better UX

User crossing tunnel for 10 seconds still appears online.

✔ More Accurate Presence

Offline only after real inactivity.

10. Real World Example

WhatsApp-like Presence

```
None
Connected now          → online

Disconnected 5 sec     → still online

Disconnected 35 sec    → offline

Reconnect              → online again
```

11. WebSocket Ping/Pong Version

Many systems use built-in WebSocket frames.

```
Server sends Ping

Client replies Pong
```

If no pong:

```
Connection considered dead
```

12. Multi-Device Presence

If user logged in on multiple devices:

```
Phone online

Laptop online

Tablet offline
```

User remains online if at least one device active.

13. Scaling Presence System

For millions of users:

```
None
Shard users across presence servers

Store heartbeat timestamps in Redis

Use pub/sub for status updates
```

14. Edge Cases

App Backgrounded

Mobile OS may pause heartbeats.

Use:

```
None
Longer timeout for mobile apps
```

Server Restart

Need reconnection logic.

Network Partition

Use timeout instead of instant disconnect.

15. Interview Answer Example

How do you detect online/offline users in chat systems?

```
None
Use persistent WebSocket connections with heartbeat ping/pong.

Clients send heartbeat every few seconds.

Presence servers update last_seen timestamp.

If heartbeat missing beyond threshold (e.g., 30 sec),

mark user offline.

This avoids frequent online/offline flickering.
```

16. Common Timeout Choices

App Type	Heartbeat	Timeout

Chat App	5 sec	30 sec
Gaming	1 sec	5 sec
Enterprise Presence	15 sec	60 sec
IoT Devices	30 sec	2 min

17. One-Line Summary

User disconnection in real-time systems is best handled using heartbeat mechanisms, where users remain online while periodic heartbeats are received and are marked offline only after a timeout window, improving accuracy and user experience.